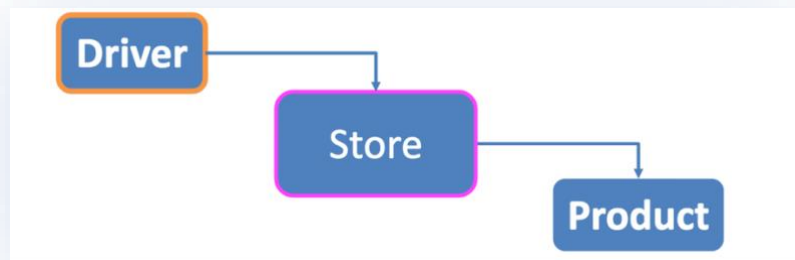


Class Diagram

All questions in this exam are based on this (one) app.



```
/** A scaled down version of a Product class ...*/
public class Product {

    private String productName = "";
    private int productCode = -1;
    private double unitCost = 0;
    private boolean inCurrentProductLine = false;

    /** Constructor for objects of class Product ...*/
    public Product(String productName, int productCode, double unitCost, boolean inCurrentProductLine) {...}

    /**...
    /** Returns the Product Name */
    public String getProductName(){...}

    /** Returns the Unit Cost */
    public double getUnitCost(){...}

    /** Returns the Product Code */
    public int getProductCode() { return productCode; }

    /** Returns a boolean indicating if the product is in the current product line */
    public boolean isInCurrentProductLine() { return inCurrentProductLine; }

    /** Updates the Product Code to the value passed as a parameter ...*/
    public void setProductCode(int productCode) { this.productCode = productCode; }

    /** Updates the Product Name to the value passed as a parameter ...*/
    public void setProductName(String productName) { this.productName = productName; }

    /** Updates the Unit Cost to the value passed as a parameter ...*/
    public void setUnitCost(double unitCost) { this.unitCost = unitCost; }

    /** Updates the boolean indicating whether the product is in the current product line or not. ...*/
    public void setInCurrentProductLine(boolean inCurrentProductLine) {...}

    /** Builds a String representing a user friendly representation of the object state ...*/
    public String toString() {...}
}

/** The store class saves all the entered products into a primitive array. ...*/
public class Store {

    private Product[] products;
    private int total = 0; //dual purpose. 1) number of items stored in array, 2) next available index location

    public Store(int numberItems) { products = new Product[numberItems]; }

    private boolean isFull() { return total == products.length; }

    private boolean isEmpty() { return total == 0; }

    /** If there is space available, add the product object, passed as a parameter, to the array. ...*/
    public boolean add(Product product) {...}

    /** This method builds and returns a String containing all the products in the array. ...*/
    public String listProducts() {...}

    /** This method returns the cheapest product in the array. ...*/
    public Product cheapestProduct() {...}

    /** This method builds and returns a String containing all the products in the array ...*/
    public String listCurrentProducts() {...}

    /** This method returns the average product price of all the products in the array. ...*/
    public double averageProductPrice() {...}

    /** This method builds and returns a String containing all the products in the array ...*/
    public String listProductsAboveAPrice(double price) {...}

    public Product get(int index){...}
    public boolean updateStockStatus(int indexNumToBeChanged){...}
}

Invalid VCS root mappings
The following directories are registered as VC
```

```

import java.util.Scanner;

/** This class runs the application and handles the Product I/O ...*/
public class Driver{

    private Scanner input = new Scanner(System.in);
    private Store store;

    public static void main(String[] args) { new Driver(); }

    public Driver() {
        processOrder();
        runMenu();
    }

    private int mainMenu(){...}

    private void runMenu(){...}

    private void changeInStockStatus() {...}

    private void processOrder(){...}

    //gather the product data from the user and create a new product object.
    private void addProduct(){...}

    //print the product (the toString method is automatically called).
    private void printProducts(){...}

    //print out a list of all current products i.e. that are in the current product line.
    private void printCurrentProducts(){...}

    //print out the average product price for all products stored in the array
    private void printAverageProductPrice(){...}

    //print out the product name that is the cheapest of those stored in the array
    private void printCheapestProduct(){...}

    //ask the user to enter a price and print out all products costing that price or more.
    private void printProductsAboveAPrice(){...}

}

```